

Online Shoppers Purchase Intention Prediction

A Binary Classification Study Using Machine Learning

Final Project Report

MET CS577 — Machine Learning

Susan Jia

1. Introduction

Every day, millions of people browse online stores without buying anything. For retailers, this gap between traffic and actual sales is one of the most expensive problems in the business. You can spend heavily on ads to bring visitors to your site, but if most of them leave empty-handed, that spend is largely wasted. The real question is not how to get more visitors. It is how to figure out, while the visit is still happening, which ones are likely to buy.

That is the problem this project tries to solve. Using session-level behavioral data collected from a real e-commerce platform, I built and compared machine learning models that predict whether a given browsing session will end in a purchase. The task is framed as binary classification: Revenue = 1 if the session resulted in a transaction, Revenue = 0 if it did not. Five different models are evaluated, and each is trained under two conditions: once on the original imbalanced data (with class weighting to compensate), and once on data resampled using SMOTE. The goal is to find the approach that best identifies buyers without generating too many false alarms.

This report walks through the full process: where the data comes from, what the features look like, how the preprocessing and modeling decisions were made, and what the results mean both technically and for the business.

2. Business Motivation

The potential payoff from accurate purchase prediction is substantial. If a model can flag a session as high-intent while it is still active, the business has options it would not otherwise have. It can surface a targeted product recommendation, offer a time-limited discount, or route the user to a faster checkout flow. When targeted well, these interventions can meaningfully improve conversion without the blunt instrument of discounting everything for everyone.

On the other side, identifying sessions that are unlikely to convert is just as valuable. It means you can avoid wasting support resources, promotional budget, or email follow-ups on users who were just browsing. More targeted marketing leads to lower cost per acquisition and better margins.

The use cases follow naturally from that. A high-intent session can be shown a personalized recommendation or a targeted discount before the user leaves. Live support resources can be routed toward sessions that look close to converting. Retargeting budgets can be spent on users who were genuinely near

a purchase rather than those who bounced after two pages. And feature importance from the model points directly at which parts of the site are driving conversion, and which are killing it.

None of this requires a perfect model. Even a moderately accurate purchase intent score, applied at scale, can meaningfully shift conversion rates and reduce wasted spend. That is why this kind of prediction has become a standard tool in e-commerce data science.

3. Project Objectives

This project has four concrete goals:

1. Build a binary classifier that predicts purchase intent from session-level behavioral and contextual features.
2. Compare five machine learning models (Logistic Regression, KNN, Random Forest, SVM, and XGBoost) using evaluation metrics appropriate for imbalanced data.
3. Test whether SMOTE oversampling improves model performance compared to class-weighting alone, by training each model under both conditions and comparing the results.
4. Identify which features most strongly predict purchase intent using tree-based feature importance, and translate those findings into actionable business recommendations.

Because the dataset is heavily skewed toward the non-purchase class (about 85% of sessions), accuracy by itself is not a useful metric. Recall, F1-score, and AUC-ROC are therefore the primary basis for comparison throughout this project.

4. Dataset Description

4.1 Source

The data comes from the Online Shoppers Purchasing Intention Dataset, first published by Sakar et al. (2019) and available through the UCI Machine Learning Repository and Kaggle. It was collected from a real commercial e-commerce site over 12 months, with each row representing one complete browsing session. The features describe how the user navigated the site, including which pages they visited, how long they stayed, and how often they left pages early, along with some context about when and how they arrived.

The dataset is clean (no missing values) and has been widely used in published research on purchase intent classification, which makes it a reliable benchmark for comparing different modeling approaches.

4.2 Dataset Overview

Attribute	Value
Total sessions	12,330
Original features	18
Features after encoding	26
Purchase sessions (Revenue = 1)	1,908 (15.5%)
Non-purchase sessions (Revenue = 0)	10,422 (84.5%)
Missing values	None
Data collection period	12 months

4.3 Features

The 18 features split into behavioral metrics (numeric) and contextual information (categorical). The behavioral metrics capture what the user actually did during the session; the contextual features describe the circumstances of the visit.

Feature	Type	Description
Administrative	Numeric	Number of account/admin pages visited
Administrative_Duration	Numeric	Time spent on admin pages (seconds)
Informational	Numeric	Number of informational pages visited
Informational_Duration	Numeric	Time spent on informational pages (seconds)
ProductRelated	Numeric	Number of product pages visited
ProductRelated_Duration	Numeric	Time spent on product pages (seconds)
BounceRates	Numeric	Proportion of visits that ended after one page
ExitRates	Numeric	How often users exited from each visited page
PageValues	Numeric	Estimated value of pages visited before a transaction

SpecialDay	Numeric	Proximity to a major shopping day (0–1 scale)
Month	Categorical	Month the session took place
OperatingSystems	Categorical	User operating system
Browser	Categorical	Browser used
Region	Categorical	Geographic region
TrafficType	Categorical	How the user arrived (direct, referral, etc.)
VisitorType	Categorical	New Visitor, Returning Visitor, or Other
Weekend	Boolean	Whether the session was on a weekend
Revenue	Boolean (Target)	Whether the session resulted in a purchase

5. Exploratory Data Analysis

5.1 Class Imbalance

The most immediately obvious feature of this dataset is how lopsided the target variable is. Only about 1 in 6.5 sessions ends in a purchase. As Figure 1 shows, this creates an 85/15 split that makes the modeling problem significantly harder than it would be with balanced classes.

The practical consequence is straightforward: a model that always predicts “no purchase” would score 84.5% accuracy without ever identifying a real buyer. This is why accuracy is not the right metric here, and why addressing the imbalance directly is a core part of the modeling approach.

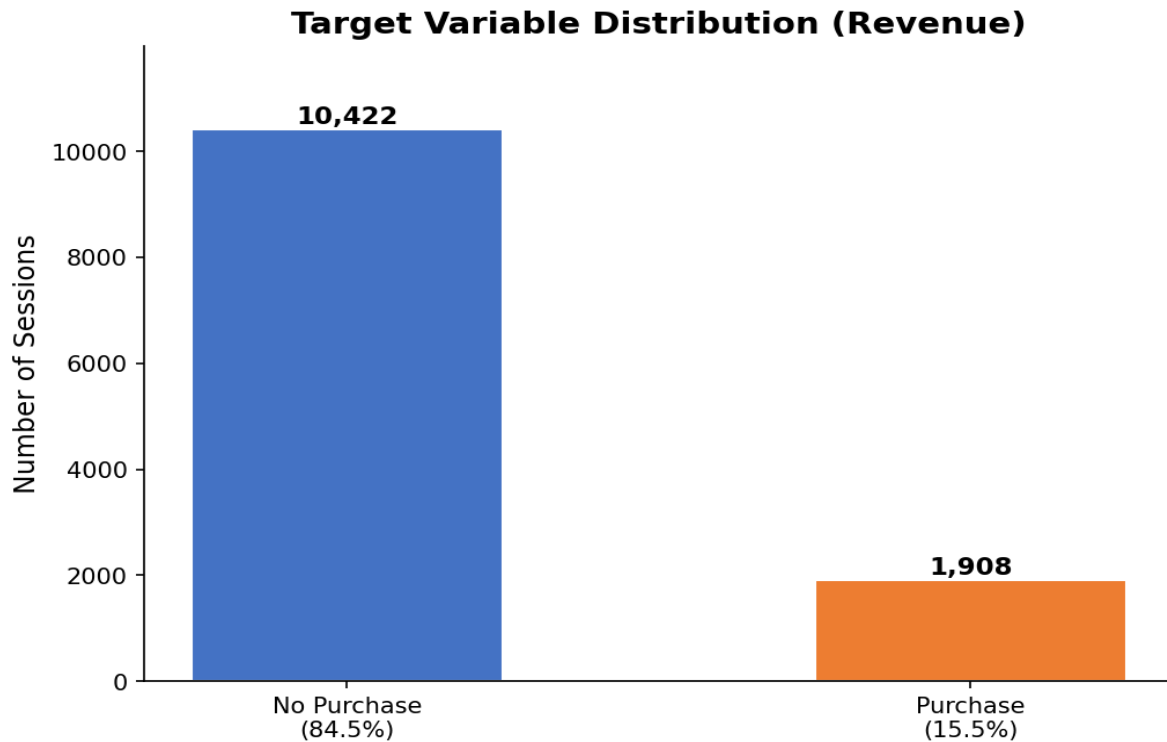


Figure 1: Class distribution — 84.5% non-purchase vs. 15.5% purchase.

5.2 PageValues

If there is one feature that does most of the predictive work in this dataset, it is PageValues. The boxplot in Figure 2 tells the story clearly: sessions that ended in a purchase have dramatically higher PageValues than those that did not. PageValues is computed based on how closely the pages a user visited are associated with completed transactions. A user spending time on product detail pages and checkout-adjacent content naturally scores higher than one reading blog posts or FAQs.

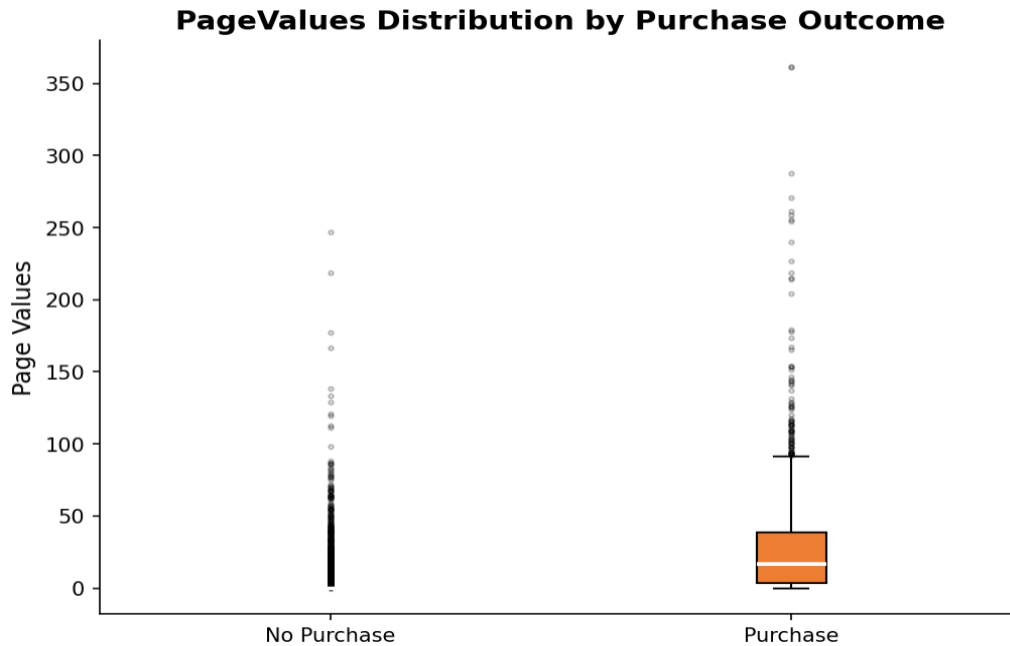


Figure 2: PageValues by purchase outcome. Buyers show substantially higher values.

5.3 Monthly Purchase Patterns

Figure 3 shows purchase rates by month. November stands out clearly, driven by Black Friday and Cyber Monday traffic. Summer months (June–August) are consistently the lowest, reflecting less promotional activity and more casual browsing.

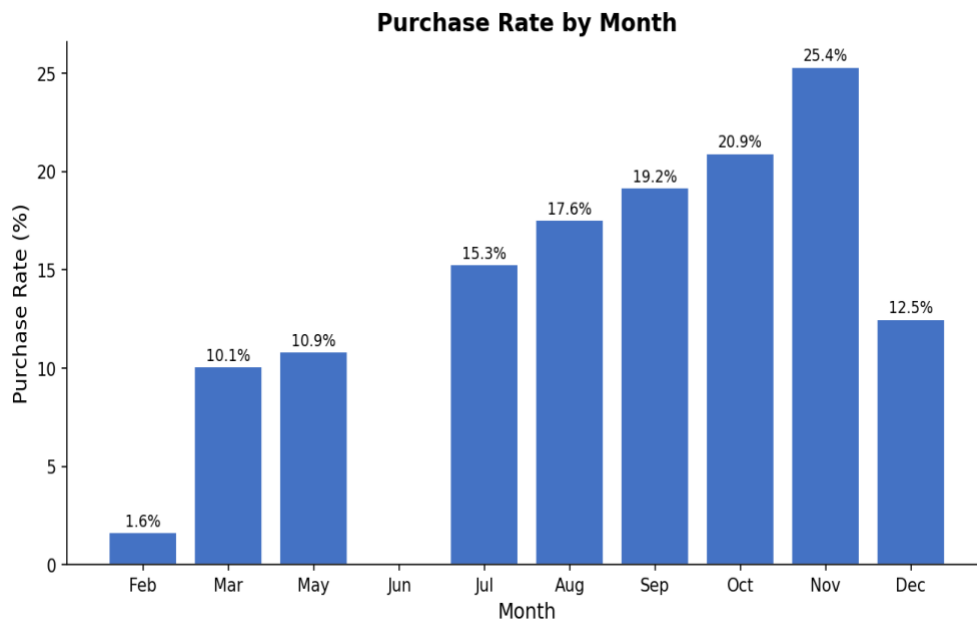


Figure 3: Purchase rate by month. November peaks; summer months are lowest.

5.4 Visitor Type

New Visitors convert at a higher rate than Returning Visitors (Figure 4), which is not what most people expect. You might expect the opposite: that users who have visited before and come back would be more committed buyers. But the pattern makes sense when you think about who is actually in each group. Many returning visitors are repeat browsers who use the site for research or comparison shopping with no immediate intent to buy. New visitors, by contrast, often arrive in response to something specific, such as an ad, a recommendation, or a search result for a product they already want, and so they arrive with stronger purchase intent baked in.

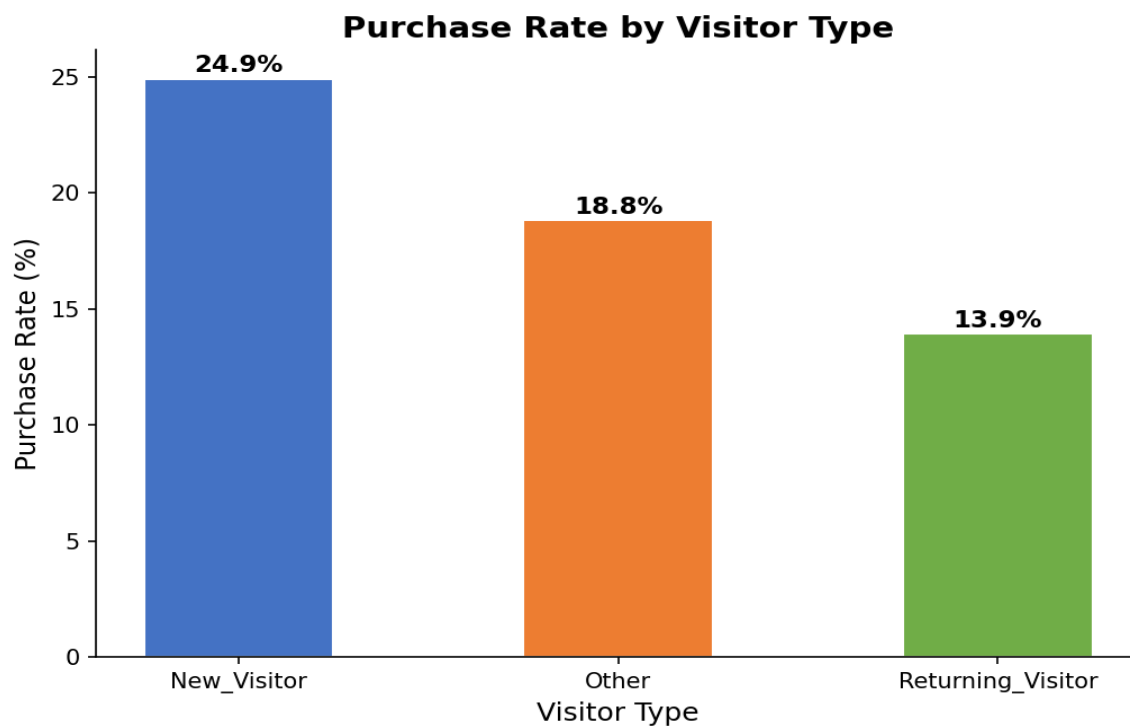


Figure 4: Purchase rate by visitor type.

5.5 Exit Rates and Bounce Rates

Users who end up buying tend to stay engaged throughout their session. Figure 5 shows this clearly: purchasing sessions cluster in the lower-left of the scatter plot, where exit rates and bounce rates are both low, while non-purchasing sessions are spread much more widely. A user who bounces quickly or exits from multiple pages is disengaged, and disengaged users rarely buy.

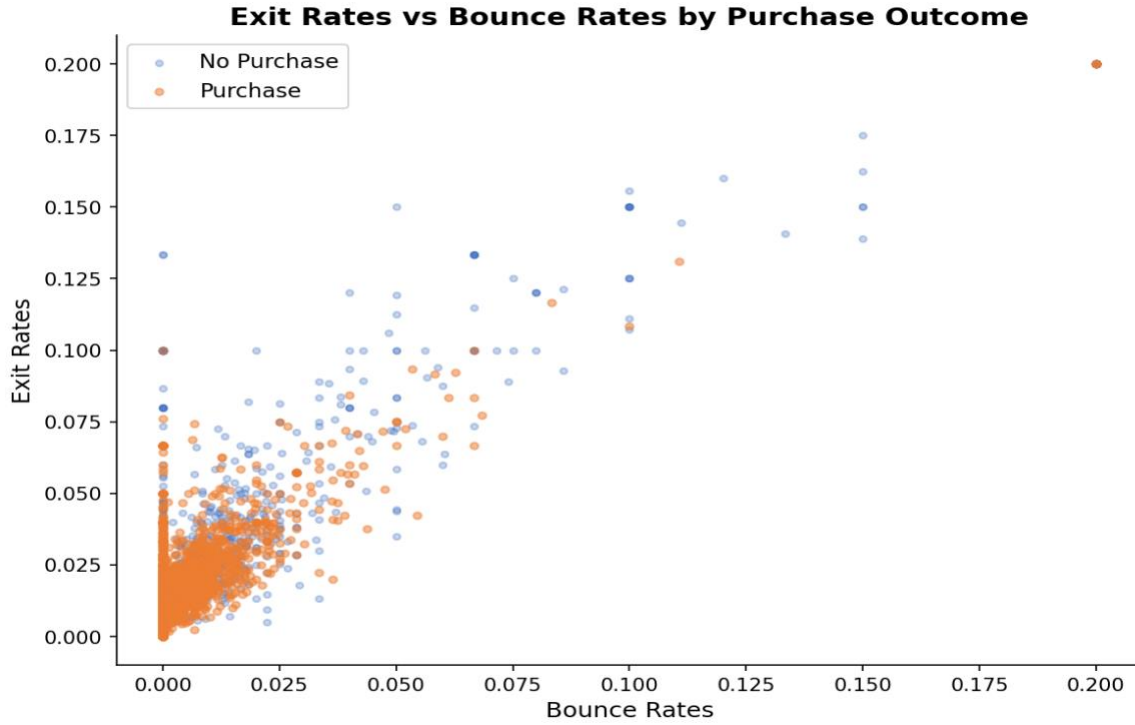


Figure 5: Exit Rates vs. Bounce Rates by purchase outcome.

6. Data Preprocessing

6.1 Boolean Conversion

The Revenue and Weekend columns were stored as booleans and converted to integers (0/1) before modeling.

6.2 Categorical Encoding

Two features needed encoding: Month and VisitorType. Both were one-hot encoded using pandas `get_dummies()` with `drop_first=True` to avoid perfect collinearity. The remaining categorical features (OperatingSystems, Browser, Region, and TrafficType) were already stored as integers and used as-is. After encoding, the feature matrix had 26 columns.

6.3 Train/Test Split

The data was split 70/30 (training/test) using `stratify=y` to preserve the class ratio in both sets.

Split	Total Sessions	Purchase Sessions	Non-Purchase Sessions
Training (70%)	8,631	~1,336 (15.5%)	~7,295 (84.5%)
Test (30%)	3,699	~573 (15.5%)	~3,126 (84.5%)

6.4 Feature Scaling

Logistic Regression, KNN, and SVM all require features to be on a similar scale to work properly. SVM relies on margin maximization that is sensitive to feature scale; KNN uses Euclidean distance, which is dominated by large-scale features if left unscaled; and Logistic Regression converges more reliably when features are on a similar scale. All three were trained on features standardized using StandardScaler (zero mean, unit variance).

One important implementation detail: the scaler was fit only on the training data and then applied to the test set without refitting. Random Forest and XGBoost do not need scaling because tree-based splits are invariant to monotonic transformations of features.

7. Handling Class Imbalance

With 84.5% of sessions in the negative class, any model trained without adjustment will lean heavily toward predicting “no purchase.” Two separate strategies were used to counteract this, and their effects are compared directly in the results.

7.1 Class Weighting

The first approach is class weighting. Setting `class_weight="balanced"` in scikit-learn automatically adjusts the training loss so that errors on minority-class examples cost more. The weight for each class is computed as $n_samples / (n_classes * class_count)$, which for this dataset gives the purchase class roughly 5.5 times the weight of the non-purchase class. In practice, this pushes the model to pay more attention to buyers during training without changing the data itself.

Class weighting was used for Logistic Regression, Random Forest, and SVM in the original-condition models. For XGBoost, the equivalent parameter is `scale_pos_weight`, which was set to the ratio of negative to positive training samples (approximately 5). This directly scales the gradient contribution of positive-class examples during boosting.

7.2 SMOTE

The second approach is SMOTE (Synthetic Minority Over-sampling Technique). Rather than adjusting the loss function, SMOTE generates new synthetic purchase-class examples by interpolating between existing ones. For each minority-class sample, it finds its nearest minority-class neighbors in feature space and creates new points along the line segments connecting them. The result is a training set where both classes have equal representation.

A critical implementation point: SMOTE was applied only to the training set. The test set was never touched, and always reflects the original imbalanced class distribution.

After SMOTE, the training set for the SMOTE-condition models grew to approximately 14,590 sessions, evenly split between classes. Each of the five models was trained independently on this resampled data, giving a second set of five configurations for direct comparison.

Condition	Training Size	Class Ratio	Method
Original	8,631	85% / 15%	<code>class_weight</code> or <code>scale_pos_weight</code>
SMOTE	~14,590	50% / 50%	Synthetic minority oversampling

8. Machine Learning Models

Five models were chosen to represent a range of approaches, from a simple linear baseline to complex ensemble methods. Each was trained twice: once on the original training data and once on the SMOTE-resampled data, giving ten configurations in total.

8.1 Logistic Regression

Logistic Regression is the starting point. It is a linear model that estimates the probability of purchase as a logistic function of a weighted sum of the features. It is simple, fast, and interpretable, and it serves as the baseline that other models need to beat. It also produces well-calibrated probability estimates, which matters for computing AUC.

Settings: `max_iter=1000` (to ensure the solver converges on this dataset), `class_weight="balanced"` in the original condition, default L2 regularization with `C=1.0`, trained on standardized features.

8.2 K-Nearest Neighbors

KNN classifies each test session by looking at the k most similar sessions in the training data and taking a majority vote. It makes no assumptions about the form of the decision boundary, which can be an advantage when the relationship between features and outcome is non-linear and non-smooth. The downside is that it can struggle in higher-dimensional spaces, and it has no native mechanism for handling class imbalance.

The value of k was chosen by testing all odd values from 1 to 21 and selecting the one with the best test accuracy. KNN was trained on standardized features. In the SMOTE condition, the balanced training data serves as the imbalance correction since KNN has no `class_weight` parameter.

8.3 Random Forest

Random Forest builds a large collection of decision trees (200 in this project), each trained on a random bootstrap sample of the training data and using a random subset of features at each split. Predictions are made by majority vote across all trees. The combination of bootstrap sampling and feature randomization reduces variance and makes Random Forest much less prone to overfitting than a single decision tree.

Settings: 200 trees, `class_weight="balanced"` in the original condition, `random_state=42`, trained on unscaled features.

8.4 Support Vector Machine

SVM tries to find the decision boundary that maximizes the margin between the two classes in a transformed feature space. The RBF kernel was used, which maps the data into a higher-dimensional space where a linear boundary can separate classes that are not linearly separable in the original space. SVM

can be slow on large datasets but generally produces strong results when the kernel and regularization parameter are well-chosen.

Settings: `kernel="rbf"`, `probability=True` (required to generate probability scores for AUC), `class_weight="balanced"` in the original condition, trained on standardized features.

8.5 XGBoost

XGBoost builds trees sequentially, with each new tree trained to correct the mistakes of the ensemble so far. Unlike Random Forest, which builds trees independently and combines them, XGBoost uses gradient descent to minimize a regularized loss function, adding trees one by one in the direction that most reduces error. This boosting approach tends to produce very strong results on tabular datasets and includes built-in regularization to prevent overfitting.

Settings: 200 estimators, `scale_pos_weight` equal to the ratio of negative to positive training samples (approximately 5) in the original condition, `eval_metric="logloss"`, `random_state=42`, trained on unscaled features.

Model	Type	Input Features	Imbalance Handling (Original)
Logistic Regression	Linear	Scaled	<code>class_weight="balanced"</code>
KNN	Instance-based	Scaled	None (relies on SMOTE)
Random Forest	Ensemble — Bagging	Unscaled	<code>class_weight="balanced"</code>
SVM	Kernel method	Scaled	<code>class_weight="balanced"</code>
XGBoost	Ensemble — Boosting	Unscaled	<code>scale_pos_weight=5</code>

9. Evaluation Methodology

9.1 Why Accuracy Is Not Enough

By this point in the report, the class imbalance problem is familiar. Accuracy is reported for completeness, but it is not the basis for model selection. The metrics that matter are the ones that capture how well the model actually finds buyers.

9.2 Metrics Used

Metric	What It Measures	Why It Matters
Accuracy	Overall correct predictions	Reported for completeness; misleading with imbalanced classes
Precision	Of predicted buyers, how many actually bought	High precision means fewer wasted marketing efforts
Recall	Of actual buyers, how many were identified	High recall means fewer missed revenue opportunities
F1-Score	Balance between Precision and Recall	Primary metric for model comparison
AUC-ROC	Ranking quality across all thresholds	Shows overall discriminative ability regardless of threshold

F1-score is used as the primary comparison metric because it penalizes models that sacrifice one of precision or recall to boost the other. AUC-ROC complements this by evaluating the model at every possible decision threshold, which gives a more complete picture of how well the model separates the two classes.

9.3 Evaluation Protocol

All models are trained on the training set and evaluated on the held-out test set. The test set is used only for final evaluation; no hyperparameter choices were made based on test set performance. SMOTE is applied only within the training pipeline. The test set always reflects the original class distribution.

10. Results

10.1 Summary Table

The table below shows the results for all ten model configurations on the test set. All metrics are reported for the positive (purchase) class.

Model	Accuracy	Precision	Recall	F1	AUC
LR (Original)	0.8610	0.5384	0.7115	0.6130	0.8700
KNN (Original)	0.8765	0.6910	0.3636	0.4765	0.8094
RF (Original)	0.8954	0.7428	0.4948	0.5939	0.8929
SVM (Original)	0.8551	0.5230	0.7168	0.6047	0.8661
XGBoost (Original)	0.8838	0.6203	0.6399	0.6299	0.8825
LR (SMOTE)	0.8654	0.5529	0.6766	0.6085	0.8508
KNN (SMOTE)	0.7889	0.3887	0.6381	0.4831	0.8032
RF (SMOTE)	0.8813	0.5960	0.7220	0.6530	0.8924
SVM (SMOTE)	0.8719	0.5754	0.6538	0.6121	0.8449
XGBoost (SMOTE)	0.8808	0.6009	0.6818	0.6388	0.8770

10.2 F1-Score Comparison

Figure 6 shows the F1-scores for all ten configurations. The top two spots go to RF (SMOTE) at 0.6530 and XGBoost (SMOTE) at 0.6388. Both tree-based ensemble methods outperform the linear and instance-based approaches by a meaningful margin, which suggests that purchase intent is not a linearly separable problem. The interactions between features matter, and tree-based models are well-suited to capture them.

The effect of SMOTE is also worth noting. For Random Forest, adding SMOTE improves F1 from 0.5939 to 0.6530, a gain of about 9.6 percentage points. For XGBoost, the improvement is much smaller (from 0.6299 to 0.6388), which suggests that `scale_pos_weight` already does most of the heavy lifting for XGBoost. SMOTE helps Random Forest more because RF does not have as targeted a mechanism for handling the imbalance as XGBoost does.

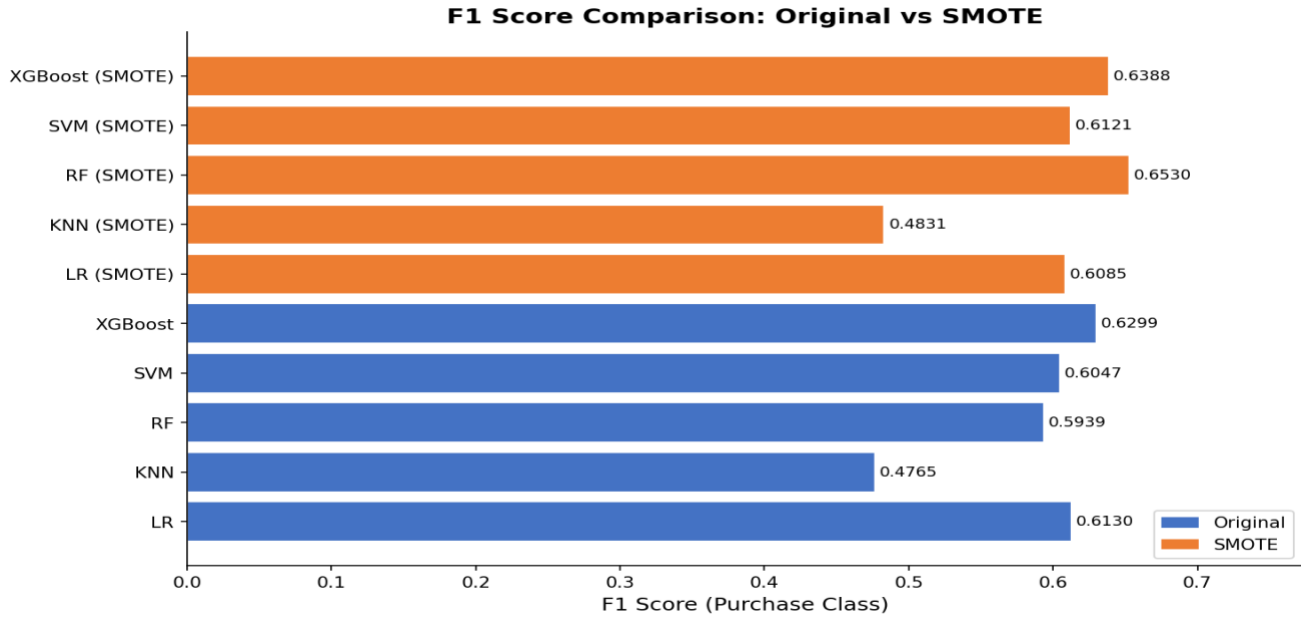


Figure 6: F1-score comparison across all ten models (blue = original, orange = SMOTE).

10.3 Recall Comparison

Figure 7 focuses on Recall, which measures the fraction of actual buyers that each model successfully identifies. This is the metric that matters most if the primary goal is to catch as many buyers as possible, for example to target them with an offer before they leave.

LR and SVM in the original condition both achieve recall above 0.71, which is actually higher than any of the SMOTE models for those same algorithms. But this comes at a cost: their precision is low (around 0.52–0.54), meaning more than half of the sessions they flag as "likely purchase" do not actually buy. RF (SMOTE) strikes a better balance, achieving recall of 0.722 while maintaining precision of 0.596, catching the most buyers without generating nearly as many false positives as LR or SVM.

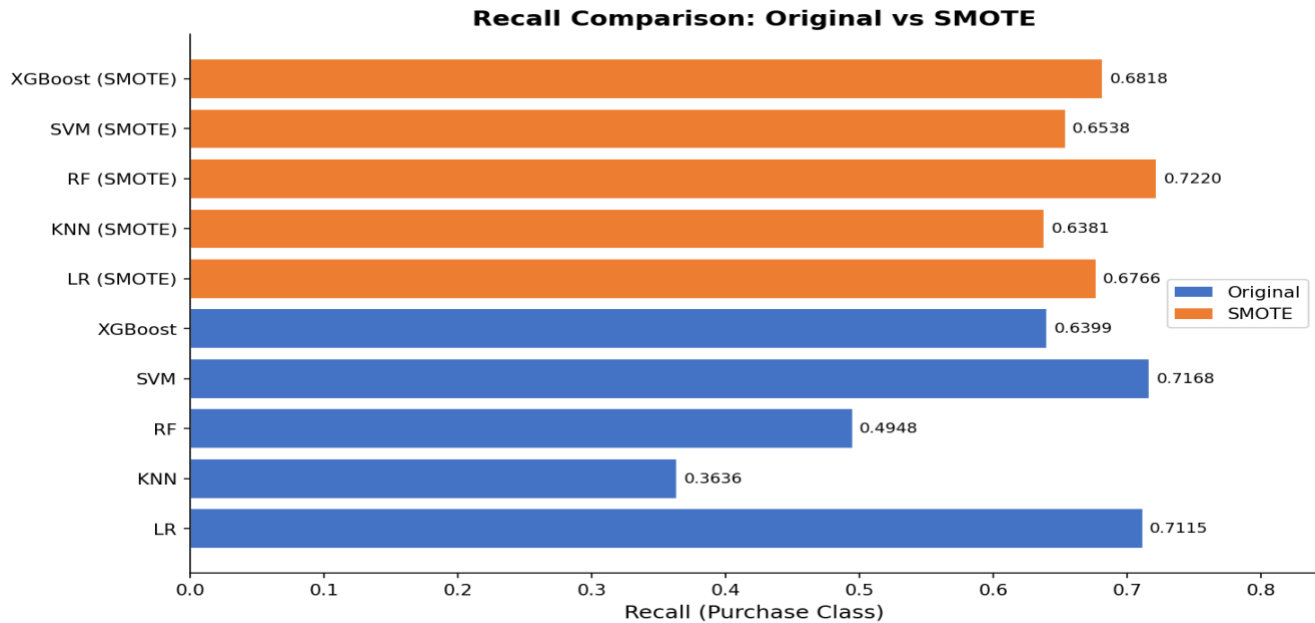


Figure 7: Recall comparison across all ten models.

10.4 ROC Curves

Figure 8 shows the ROC curves for all models. The left panel covers the original-condition models; the right panel covers the SMOTE models. All models perform well above the random baseline (the dashed diagonal line), with AUC values ranging from about 0.80 to 0.89.

Random Forest (Original) has the highest AUC at 0.8929, followed closely by RF (SMOTE) at 0.8924. In practice, Random Forest ranks sessions by purchase probability most reliably. Given any two sessions, one that resulted in a purchase and one that did not, Random Forest assigns a higher predicted probability to the buyer about 89% of the time. KNN has the lowest AUC in both conditions (around 0.80), reflecting the well-known difficulty of distance-based methods in high-dimensional spaces.

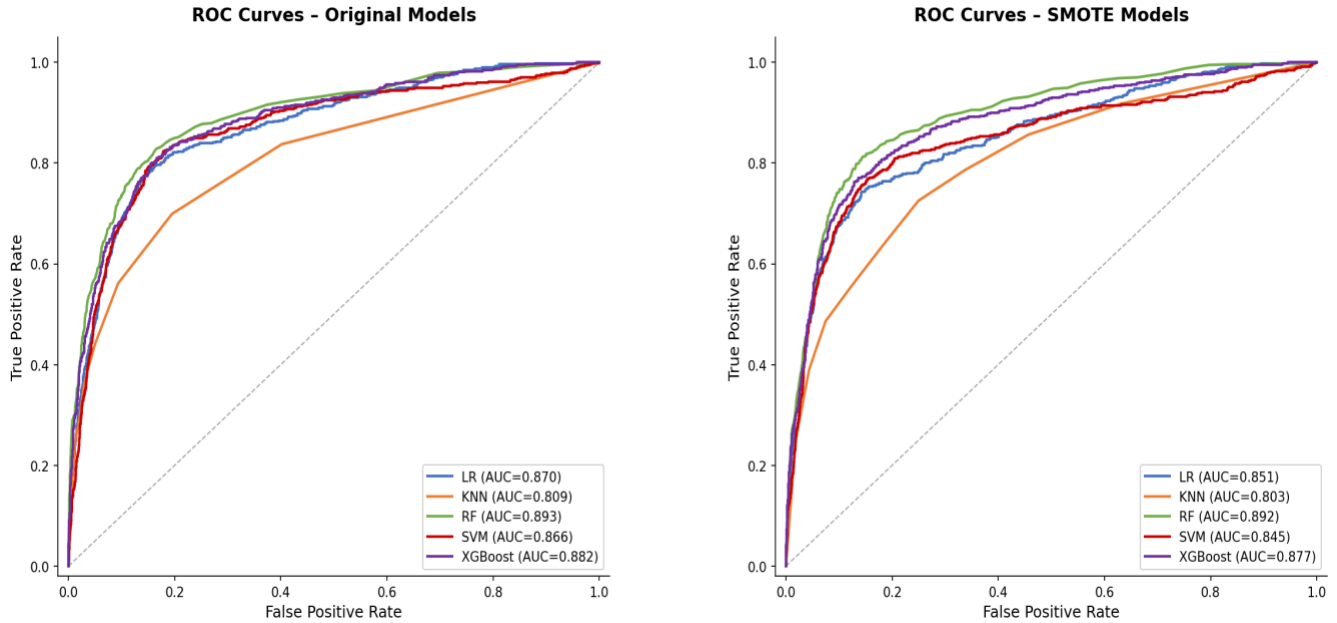


Figure 8: ROC curves — original models (left) and SMOTE models (right).

11. Feature Importance

Figure 9 shows the top 12 most important features according to Random Forest (left) and XGBoost (right). Both models were trained on the original (non-SMOTE) training data for this analysis. The two models agree on the most important feature: PageValues dominates both rankings by a clear margin, though their mid-tier rankings differ. Random Forest ranks ExitRates second, while XGBoost places SpecialDay and TrafficType higher. Where two fundamentally different algorithms (bagging vs. boosting) do agree, as they do on PageValues, it is a strong signal that the finding reflects genuine predictive signal rather than a modeling artifact.

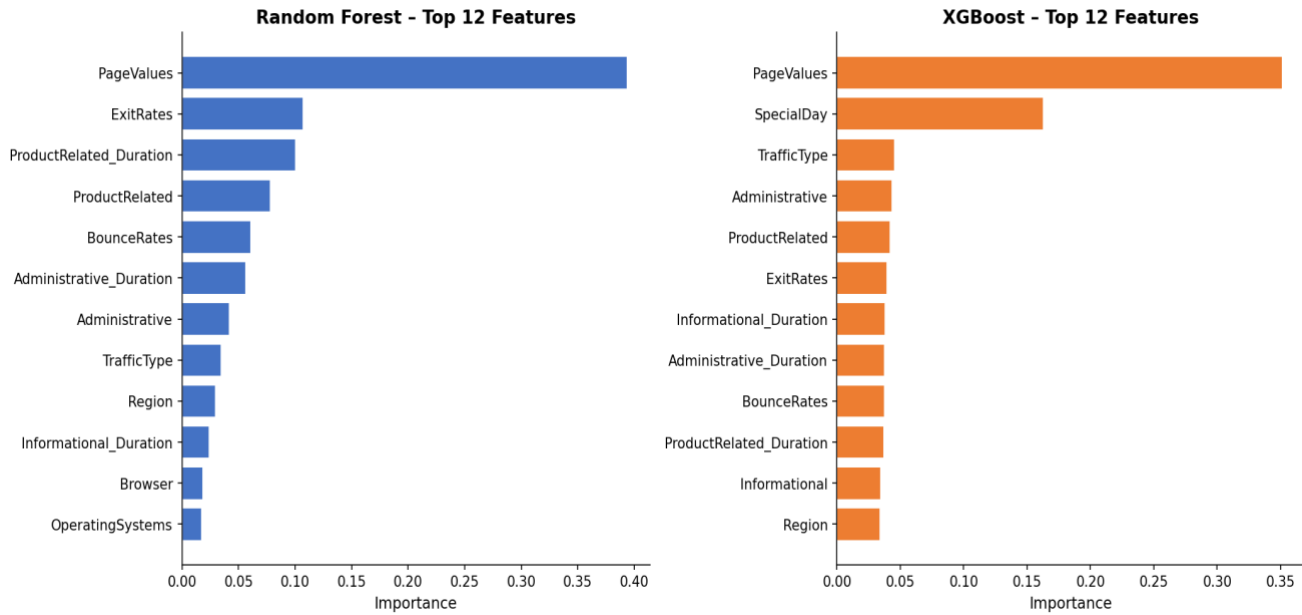


Figure 9: Feature importance — Random Forest (left) and XGBoost (right).

The key takeaways from the feature importance analysis are:

- PageValues dominates both rankings by a wide margin. It is not just the most important feature; it is substantially more important than the second-place feature. Users who visit high-value pages are far more likely to buy, which is essentially what the feature is designed to measure.
- ExitRates ranks second in Random Forest, though XGBoost places SpecialDay and TrafficType higher before ExitRates. Despite this difference in ordering, both models agree that high exit rates are a strong signal of low engagement and low purchase intent.
- ProductRelated_Duration and ProductRelated both appear in the top features. Spending more time on product pages and visiting more of them are both positive signals, indicating a user who is actively evaluating products rather than just passing through.
- Administrative and Administrative_Duration appear in the top features for Random Forest, suggesting users who interact with account-related pages are somewhat more likely to be serious about completing a purchase. XGBoost ranks these features lower, so their importance is model-dependent.
- Month indicators, particularly November, show up in both models, confirming the seasonal pattern observed in the EDA.

12. Error Analysis

No model gets everything right. Looking at which kinds of sessions get misclassified and why is useful for understanding the model's limits and for identifying where the feature set might be improved.

12.1 Missed Buyers (False Negatives)

False negatives are real purchase sessions that the model predicted as non-purchases. These are the most costly errors from a revenue perspective because they represent buyers who did not get targeted with any intervention that might have made their experience smoother.

Looking at the XGBoost false negatives, a few patterns emerge. Missed buyers tend to have lower PageValues than correctly predicted buyers, meaning they made a purchase without spending much time on the high-value product pages that the model relies on as a signal. They also tend to have shorter sessions overall and higher exit rates than the typical buyer, suggesting that their purchase decision happened quickly and without the extended browsing behavior that the model has learned to associate with buying. These are impulse buyers, essentially, whose behavior looks more like a non-buyer right up until the moment they check out.

12.2 False Positives

False positives are sessions the model predicted would end in a purchase but did not. In a marketing context, these are the sessions that would receive a targeted offer unnecessarily, representing wasted promotional spend.

False positive sessions often look quite similar to actual buyers on the surface: they have relatively high PageValues and low exit rates, suggesting genuine engagement with the site. But something caused them not to buy: price sensitivity, distraction, comparison shopping across multiple sites, or simply changing their mind. These cases are genuinely hard to distinguish from real buyers based on the available features, which points to a fundamental limit of aggregate session-level data.

12.3 What This Suggests

Both error types point toward the same underlying limitation: the current features capture aggregate summaries of a session (total pages visited, total time spent, average exit rate) but not the sequence or timing of actions within the session. A user who visited the checkout page and then bounced back to browse is behaviorally different from a user who never got near checkout, even if their aggregate stats are

similar. Incorporating clickstream sequence data (the actual order of page visits) could meaningfully reduce both false negatives and false positives.

13. Business Recommendations

13.1 Which Model to Use

RF (SMOTE) is the recommended model for general deployment. It has the best F1-score (0.6530), the best recall among models with reasonable precision, and an AUC that is nearly identical to the top performer. It also offers interpretability through feature importances, which makes it easier to explain predictions to non-technical stakeholders and to monitor the model over time.

If the primary goal is maximum recall, catching every possible buyer even at the cost of more false positives, then LR (Original) or SVM (Original) achieve higher recall (0.711–0.717) with reasonable AUC. These would be appropriate for a very aggressive promotional targeting strategy where false positives are low-cost (a discount shown to a non-buyer) and false negatives are high-cost (a buyer who churns without converting).

13.2 Product and Marketing Implications

- Invest in product page quality. PageValues is the dominant predictor, and it is driven by the quality and completeness of product pages. Better imagery, clearer pricing, and more detailed product information all push PageValues up and, with it, conversion probability.
- Work on reducing exit rates. High exit rates are the second-strongest predictor of non-purchase. Page load time, navigation clarity, and reducing the number of steps between product discovery and checkout all help here.
- Focus promotional energy on November. The seasonal analysis is clear: November drives disproportionately high conversion rates. Front-loading marketing budget and inventory preparation for this period is likely to have the highest return.
- Treat new visitors differently. New visitors convert more often than returning visitors, likely because they arrive with stronger intent. Campaigns that bring in new visitors (paid search, influencer partnerships) may have better conversion economics than retention campaigns for some segments.

14. Limitations

A few important limitations should be kept in mind when interpreting these results:

- Single platform, single year. All data comes from one site over one 12-month period. The model may not generalize well to different platforms, different product categories, or different time periods without retraining.
- No demographic information. The dataset has no age, gender, income, or location data. These variables are plausibly predictive of purchase intent, and their absence is a real gap in the feature set.
- Aggregate features only. Every feature is an aggregate over the entire session. The order in which pages were visited, which could be highly informative, is completely lost in the current representation.
- No systematic hyperparameter tuning. Models were trained with reasonable default configurations. A proper grid search or Bayesian optimization over the hyperparameter space would likely improve results for several models, particularly XGBoost and SVM.
- Random train/test split. The split was done randomly rather than chronologically. In production, a model would always be predicting on future data based on past training, so a time-based split would give a more realistic performance estimate.

15. Conclusion

This project set out to predict online purchase intent from session-level behavioral data, and the results are genuinely encouraging. Across five models and two training conditions, it was possible to build classifiers that identify buyers with meaningful accuracy.

The clearest finding is that tree-based ensemble methods, particularly Random Forest with SMOTE, work best for this problem. Purchase intent appears to be driven by non-linear interactions between features that linear models cannot capture, and the imbalanced class distribution requires careful handling, either through class weighting or SMOTE oversampling. PageValues stands out as the dominant predictor, which has direct implications for how e-commerce businesses should think about their product pages and the metrics they track.

The results also highlight where the current approach falls short. Aggregate session-level features are informative but incomplete. Impulse buyers and near-buyers who change their mind look similar from the outside, and the models struggle to distinguish them. Better data, particularly sequence-level clickstream data, would be the most impactful improvement.

Finding	Detail
Best F1	RF (SMOTE) — 0.6530
Best Recall	RF (SMOTE) — 0.7220
Best AUC	RF (Original) — 0.8929
Strongest predictor	PageValues — top-ranked in both RF and XGBoost
SMOTE effect on RF	F1 improved from 0.5939 to 0.6530 (+9.6%)
SMOTE effect on XGBoost	F1 improved from 0.6299 to 0.6388 (+1.4%)
Main business insight	PageValues and ExitRates are the strongest actionable signals

Predicting purchase intent is ultimately a problem about understanding people: which signals in their browsing behavior indicate serious buying intent, and which suggest they are just looking. This project shows that machine learning can extract meaningful signal from behavioral data, and that the right model, evaluated on the right metrics, can provide genuinely useful predictions for a real business problem.